

VSDL to KYPO(V2K) technical documentation*

Alexander Kováč

Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií
`xkovaca2@stuba.sk`

29. April 2026

*Bachelor project, ac. year 2025/26, supervisor: Michal Kuklovský

Contents

1	About V2K	4
1.1	Quickstart	4
2	Architecture	4
2.1	Preprocessor	5
2.2	Specifier	5
2.2.1	Generating assertions	5
2.2.2	Declared terms	6
2.2.3	Implications	6
2.2.4	Environmental constraints	6
2.3	Solver	6
2.3.1	Finding solution	7
2.3.2	Formatting solution	7
2.3.3	Assigning IP addresses	7
2.4	Translator	7
2.4.1	Topology definition	8
2.4.2	Provisioning definition	8
2.4.3	Configuring software	8
2.5	Files structure	9
3	Syntaxes	11
3.1	VSDL-lite	11
3.1.1	VSDL-lite syntax	11
3.1.2	Allowed values	15
3.1.3	Comparison with original VSDL	15
3.2	Preprocessed VSDL-lite	17
3.2.1	Standardized formatting	17
3.2.2	Sub-statement bracketing	18
3.2.3	Vulnerability expansion	18
3.3	SMT specification	18
3.3.1	Term names	19
3.3.2	Version encoding	20
3.3.3	Summary of statement to term relations	21
3.4	Solution	22
3.4.1	Software properties	22
3.4.2	User permissions	23
3.4.3	Connected nodes	23
3.4.4	Firewall rules	24
3.4.5	Summary of properties	25
3.5	Sandbox definition	26
3.5.1	Topology definition	26
3.5.2	Provisioning definition	26

4	Program characteristics	27
4.1	Program options	27
4.1.1	Running program without options	28
4.2	Implicit behaviour	28
4.3	Use cases	29
4.3.1	Automatic generation	29
4.3.2	Simple provisioning	29
4.3.3	Verifying scenario	29
4.4	Time complexity in practice	30
5	Limitations	30
5.1	Debugging difficulty	31
5.2	Configuring software	31
5.3	Program performance	31
5.4	Inherited limitations	32
5.5	Internet access	32
5.6	Linux specific	32
6	Setup process	33
6.1	Installation guide	33
6.2	Environmental limits	33
6.2.1	Generic limits	34
6.2.2	Complex limits	35
6.3	Base boxes	36
6.3.1	Base box properties	36
6.3.2	Adding base box	37
6.4	Software configuration scripts	37
6.4.1	Selection process	38
7	Example of usage	38
7.1	Writing scenario	38
7.2	Compiling scenario	40
7.2.1	Writing the software configuration script	41
7.3	Modifying output	43
7.3.1	modifying the provisioning	43
7.3.2	populating the files	43
7.4	Launching sandbox definition on KYPO CSC	43
7.4.1	Problematic steps	44
7.4.2	Modifying Vagrant file	44
7.4.3	Modifying pre-configuration playbook	45
7.4.4	Accessing the scenario	46
7.5	Launching sandbox definition on KYPO CRP	46
7.5.1	Reconfiguring V2K for CRP	46
7.5.2	Deployment	46
7.5.3	Accessing the scenario	47
7.6	Ways to improve the example scenario	49
7.6.1	Adding more realistic context	49
7.6.2	Making universal netplan pre-configuration playbook	49
7.7	Scenario walk-through	50
7.8	Cleanup process	52

1 About V2K

The VSDL to KYPO or V2K for short is program designed to translate a high abstract scenario written in VSDL based syntax to more precise machine readable format. The program partially verifies the scenario and catches most common errors before time consuming scenario deployment process on the KYPO CRP or CSC.

The program is written in python and uses only 3 non standard libraries, CVC5, ndvlib and PyYaml. CVC5 is used to verify the scenario against environmental constraints. The ndvlib translates the specified CVEs to software and OS statements.

The output is a sandbox definition consisting of provisioning definition and topology definition. These can be deployed on a cloud on the KYPO cyber range platform or locally on the Cyber sandbox creator. Depending on the base box used and target configuration, additional configuration may be required.

The aim of this program is to improve the development process of cybersecurity scenario by simplifying standard tasks, precise software configuration is incompatible with high abstract nature of VSDL. For such a purpose, it is still required to take generated sandbox definition and modify it.

1.1 Quickstart

Majority of documentation is focused on explaining the inner workings of V2K. For quickly setting up and using V2K, see [Installation guide](#) and [Example of usage](#). And the [VSDL-lite](#), to learn the language used to define scenarios.

2 Architecture

The translation process of VSDL based syntax to KYPO sandbox definition can be broken down to 4 stages:

1. stage - preprocessor
2. stage - specifier
3. stage - solver
4. stage - translator

The stages can be run separately or in series depending on program arguments; see [program options](#) for more details. This separation simplifies the debugging process and enables a more predictable behaviour to be forced.

The preprocessor and translator stages behave deterministically and can be run repeatedly without risk of changing the generated output. The preprocessor has a low performance cost in general, but when injecting new vulnerability for the first time, it will incur a one-time performance penalty.

The translator's deterministic characteristic is useful when recreating the same scenario multiple times, even if the generated sandbox definition is deleted or modified; as long as the [cache](#) is not cleared, it can be easily regenerated.

The specifier and the solver do have inherent randomization present, and running them repeatedly can change the generated scenario. The solver usually also takes up most of the program runtime to verify the scenario.

2.1 Preprocessor

The preprocessor is the outermost stage that takes in scenario definition written in [VSDL based syntax](#). It handles simple preprocessing to [preprocessed VSDL](#) and vulnerability injection.

The preprocessing process consists of these steps:

1. read file
2. extract objects
3. add brackets around logic operations
4. format units
5. translate vulnerabilities
6. save preprocessed VSDL

Vulnerability injection is the most complex task at this stage. To translate CVEs from suffer statements; preprocessor uses `nvdlib` to query NVD and get vulnerable configurations. The vulnerable configuration is then translated into OS and software statements.

The translated CVEs are also [cached](#) for later use. When the preprocessor is tasked with translating the already cached CVE vulnerability next time, it will use the cached statement to skip querying, this also means that V2K is capable of working fully in offline mode, as long as no new vulnerabilities are introduced.

2.2 Specifier

The specifier is the second stage in the V2K pipeline, responsible for generating [SMT specification](#) from [preprocessed VSDL](#). It handles randomization, software versioning, and environmental limits.

The specifying process consists of these steps:

1. read file
2. generate assertions
3. format implications
4. get environmental constraints
5. write the SMT specification

2.2.1 Generating assertions

Generating assertions is the most comprehensive task at this stage. The preprocessed VSDL is handled line by line. The statements are processed recursively, first logical operations are processed, until the sub-statements no longer have logical operations.

The simple statements that do not use logical operations are then handled in case by case manner, where for each keyword, corresponding part of code processes the simple statement. This process is done by script called simple statement resolver, and as a byproduct, new declared terms are added and implications generated.

2.2.2 Declared terms

The declared terms represent the device and network properties in SMT syntax. During assertion generation, new terms are declared when required to test assertions. These are collected into a set and declared before asserts to avoid SMT errors. The terms follow a simple naming scheme and have their type determined later based on the prefix as defined in [term.types](#).

Certain terms are optional and are defined only once a related statement is parsed. However, some are required and must have value, like flavour and base box. These are automatically declared when a new node or network is parsed.

2.2.3 Implications

Because of support for logical operations, the trueness of statement is not always known at this stage and may in fact be nondeterministic, based on chosen solution in the solver stage. For complex statements which rely on other statements, implications are used.

For example, a user having certain access rights to a certain file relies on the existence of the file and the user. In such a case, an implication is assigned to this statement and later added to the SMT specification. The main benefit of this approach is the better readability of the SMT specification and higher flexibility in new features.

2.2.4 Environmental constraints

The environmental constraints do not originate from the supplied scenario definition. Instead, they are generated to ensure that the scenario fits the configured limits, defined in the [data folder](#). These limitations are based on the hardware and software available.

2.3 Solver

The solver is responsible for finding a valid configuration based on the [SMT specification](#). The solver uses the CVC5 library to try to solve the provided assertions and either find the solution or determine terms and statements that prevent it from being impossible.

Once the solution is found, the solver handles formatting and standardizing. The result is a [standardized valid configuration](#) that can be used to deterministically generate the sandbox definition by the translator.

To generate a valid configuration from the SMT specification, the solver follows these steps:

1. read the SMT specification
2. find solution
3. assign terms to device and node properties
4. format the solution
5. assign IP addresses
6. separate hosts from routers

7. assign firewall rules
8. save the valid configuration

2.3.1 Finding solution

This subtask is usually the most time consuming in the whole program, often times taking majority of runtime; it is, however, required to ensure proper verification against environmental constraints and itself. The CVC5 provides solver and debugging capabilities which are used to search for solution and print problematic terms and assertions if no solution was found.

2.3.2 Formatting solution

The solution found by CVC5's solver is not easily machine readable and requires proper formatting for the latter use. This is done in multiple steps; first, the results of device and network properties are parsed to the correct data types and then properly encapsulated.

2.3.3 Assigning IP addresses

Most of the device and network properties are present in solution, but in case of unspecified ip addresses, the solver stage handles their assignment non-deterministically. This process first assigns specified IP addresses to avoid generating conflicting assignments and the semi-randomly picks from remaining unused IP addresses.

The process is not without compromise. To avoid complicating the IP assignment, the unused but mentioned IP addresses cannot also be used. This can lead to false errors, claiming that no address space was left to assign IP address. This can be mitigated by always providing an alternative statement that uses the mentioned address or by increasing the network size.

2.4 Translator

The translator is the last stage of program and is responsible for translating the [valid configuration](#) into the machine readable [sandbox definition](#). The generated sandbox definition can then be deployed on KYPO CRP or CSC.

The deterministic nature of this stage ensures easy regeneration of the sandbox definition in case of loss of the first output, the intermediary valid configuration remains in cache until it is written over by the solver stage. The generated sandbox definition can differ, based on the program configuration present in the [data folder](#). This makes it viable to change custom software configuration scripts and test the effects with the rest of original configuration being used.

The translation process is carried out in these steps:

1. read valid configuration
2. define topology
3. save topology definition
4. generate the provisioning definition
5. save the provisioning definition

2.4.1 Topology definition

The topology definition is generated first based on the supplied valid configuration. All required properties present in the [topology definition](#) are determined and influenced by the user scenario. This process is relatively straightforward, with no real chance of failing, as long as the input originates from previous stages. No additional topology verification is required in this stage.

2.4.2 Provisioning definition

Generating the provisioning definition is a much more complex process than generating the topology definition. The software and device configuration requires a much less standardized approach. For this purpose, the [provisioner script](#) is called.

The provisioner script generates the provisioning definition for all routers and hosts based on the valid configuration. To configure the device to the required state, the Ansible is used. The Ansible luckily provides somewhat standardized method of automatic configuration, and the provisioner uses the Ansible built-in modules to generate provisioning scripts, which will be called and executed on devices during provisioning.

At the start of provisioning scripts, base box specific Ansible scripts are placed to modify base box if required. This step is not always required and is based on the optional property of the entries in [base_boxes.yml](#). These serve more for extending compatibility with older or misconfigured base boxes, which may require small configuration, like updating keyring.

2.4.3 Configuring software

The exception to the automatically generated Ansible scripts are software deployment tasks which need to be defined manually. More precisely, versioned software configuration tasks need to be defined manually, and the unversioned software statements will be translated automatically with the use of the Ansible `app` module.

Versioned software statements are most often used to describe vulnerable configurations and are usually not available in public repositories for security reasons. In these cases, it is often necessary to download, build, and compile a specific version of software.

To our knowledge, there is no solution that could reliably accept the name and version of the software to download, install, and configure vulnerable service for any reasonably sized subset of software products. The V2K approach to this problem is to rely on the user to provide manual software configuration.

There are two ways for the user to handle the versioned software configuration. The first option is for software to be preconfigured directly on the [base box](#), in which case; the base box entry in the [base_boxes.yml](#) contains the name of the software together with the version in the optional `preconfigured_software` property.

The second option is for the user to define the configuration script for the required version of the software. The Ansible configuration scripts are stored in [nd](#) contain constraints that determine on which base boxes and operating systems they may be deployed. These constraints are already taken into account by the solver as environmental limits.

2.5 Files structure

The main program consists of 3 directories: scripts, data, and cache. Additionally, the pycache folder will be automatically generated after running program. The other folder likely to be generated is the default output folder, if the user does not specify the different output in the [options](#).

The additional files are the example folder and documentation. These are not strictly needed to use the V2K program, but they provide valuable insights and explanation to new users.

./

- main.py - program executable
- input.vsd1 - default input file
- requirements.txt - file containing the required libraries by V2K
- V2K_technical_documentation.pdf - documentation of V2K (this file)

./cache/

The intermediary files, which are automatically generated during scenario compilation.

- vulnerabilities.json - cached VSDL statements for vulnerable configuration
- preprocessed.vsd1 - [standardized VSDL](#) format produced by preprocessor and used by specifier
- assertions.smt2 - [SMT specification](#) produced by specifier and used by solver
- solution.json - [valid configuration](#) produced by solver and used by translator

./data

The files describing environment constraints.

- limits.json - json file containing environmental limits, the constraints limiting acceptable values
- base_boxes.yml - yaml file containing additional info to [base boxes](#)
- softwares.yml - taml file containing configuration scripts for different software and [version combinations](#)

./examples

The collection of examples generated by V2K. Each example entry consists of copied data, cache and output folder.

- vuln-apache-CRP - the example scenario [deployable on CRP platform](#)
- vuln-apache-CSC - the example scenario [deployable on CSC platform](#)
- simple-CSC - simple example scenario intended for fast deployment and showcase of simple configuration capabilities of V2K

./scripts/

The scripts folder contains all scripts used by the main script.

- term.types.py - script declaring types of terms used in [SMT specification](#)
- utils.py - script consisting of helper functions
- stage1.py - 1st stage of program - preprocessor
- stage2.py - 2nd stage of program - specifier
- stage3.py - 3rd stage of program - solver
- stage4.py - 4th stage of program - translator
- vulnerability_injector.py - script responsible for querying, parsing and caching CVEs
- simple_statement_resolver.py - script to resolve non-logical statements
- enviromental_constraints.py - script to generate SMT assertions required to keep solution compatible with specified environment
- software_versioning.py - script for managing versioned software
- provisioner.py - script for generating provisioning definition

./output/

The default output folder of V2K. Containing the topology definition and sandbox definition.

- topology.yml - topology definition
- sandbox - sandbox definition
 - provisioning - provisioning definition based from KYPO sandbox definition format
 - * group_vars
 - * host_vars
 - * roles
 - * playbook.yml

3 Syntaxes

The V2K syntax can be broken into 5 different formats with 4 stages transforming the initial VSDL based syntax gradually into the final sandbox definition. This process is gradual and every stage accepts only one format as input and generates output in next format.

3.1 VSDL-lite

For defining the scenario, the V2K uses VSDL-lite language. The VSDL-lite is high level abstract language strongly based on VSDL, but modified to better suit capabilities and needs of KYPO deployment platform.

3.1.1 VSDL-lite syntax

The VSDL-lite defines the scenario and the nodes and networks belonging to it. Nodes and networks are made up of statements that describe the device or network configuration. The VSDL requires the scenario, node, and network declarations to be on their own separate lines, isolated from other keywords. However, statements can be stretched over multiple lines to improve readability.

```
scenario scenario_name {
    node node_name {
        ...statements...
    }
    network network_name {
        ...statements...
    }
}
```

Figure 1: Example of top level VSDL-lite syntax

The statements can be either logical statements consisting of one or more sub-statements or simple statements directly describing node or network configuration. symbol or logical operators. Logical statements can use operators listed below with the same order of priority.

```
not statement1;
statement1 and statement2;
statement1 xor statement2;
statement1 or statement2;
```

Figure 2: VSDL-lite logical operators

The top level statements are terminated by semicolon ; . The sub-statements are separated by logical operators or brackets. The logical statements can be chained indefinitely with the exception of **not** operator. If multiple logical operators interact with the statement, they are resolved in order from first to last logical operator. Meaning **not** first, followed by **and**, **xor**, **or**.

Network simple statements

The networks support 6 simple statements to describe them. These are ip range, connected nodes and firewall rules. Together, they fully describe the fundamental networking behaviour as well as simple firewall rules.

```
ip range from Addr to Addr;
node Id is connected;
node Id has IP Addr;
firewall blocks ( Port Num | IP Addr );
firewall forwards ( IP Addr to Addr | Port Num to Num );
is user accessible;
```

Figure 3: VSDL-lite network statements

The ip range statement describes the first and last ip addresses of the network. The first address can be either the first valid host address or the network id. The second address must be either the broadcast address or the first bigger ip that does not belong to the network.

The **node is connected** and **has IP** statements are used to state that the node is connected to the network, and potentially specify the node's ip address in that network. The ip address is checked and verified during the [solver stage](#).

Node simple statements

The nodes support a much larger collection of statements, from simple declaratory statements influencing one numerical value to more complex statements describing users, filesystem objects, and software configurations.

The text values in node statements must be a valid string, with no white spaces or a string enclosed in quotation marks. In case of quotation mark enclosed string it is important, that no additional quotation marks are present in the string.

Is statements

The most numerous of simple statements are those with the operator **is** between node property name and value constraint. The **is** statements follow simple syntax and have 3 main variations as listed below.

```
Attribute is same as Id;
Attribute is Value;
Attribute is Comparison_Op Num Unit;
```

Figure 4: VSDL-lite node is statements

The **attribute** describes which property is influenced by the statement. Depending on the type, different **is** statements can be used. The **is same as Id**

statement can be used for all types of properties, regardless of whether they are numeric or text based.

The second **is** statement variant is the simplest variant. It is used on the text based node properties, such as type, flavor, basebox or OS. The small difference being the OS, unlike other simple statements, it can be extended by version hints in the same fashion as [software statements](#).

Statements with **Comparison_Op** can be used on numeric properties such as memory and disk. They use **Comparison_Op** to influence the numerical value. The Units are also taken into account and translated based on their prefix. The V2K program supports standard SI prefixes (K,M,G,T,P).

Comparison_Ops		Math op
equal	to	=
larger	than	>
smaller	than	<
faster	than	>
slower	than	<
at	least	>=
at	most	<=

Table 1: VSDL-lite comparison operators

File and directory statements

The next statement in VSDL-lite is **contains** statement. This statement is used to influence the directories and files created. This statement also allows extension to specify owner of created file or directory.

```
contains ( file | directory ) Path;  
contains ( file | directory ) Path owned by User
```

Figure 5: VSDL-lite node contains statements

User statements

To describe node users and their permissions, the V2K uses 5 different statements. The owned by statement has been already mentioned [above](#).

```
exists user User;  
exists user User with password Password;  
contains ( file | directory ) Path owned by User;  
user User can ( read | write | execute ) Path;  
user User ( can sudo | is admin );
```

Figure 6: VSDL-lite node user statements

The **can** statements describe **User** in relation to the **file** or **directory** and their access rights. The can statement also has edge case to give administrator rights to the **User**.

The **exists** statement is used to declare user with optional extension to choose password. Without the password extension, user password is set same as username.

Software statement

The software statement specifies the software to install on the node and optionally its version. The statement has 3 variants with the 2nd and 3rd variants adding ability to add version constraints of specified software. Version hints are also used in **OS is** statements, mainly to improve [vulnerability injection](#).

```
mounts software Software;
mounts software Software-Version;
mounts software Software Version_hints*;
Version_hints = ( on | above | from | below | to )
Version
```

Figure 7: VSDL-lite node software statements

The software statements provide the most detailed device configuration of all statements. The simplest variant ensures that the desired software package is installed on the node after provisioning. The extension of the software statement in the form of **version** and **version hints** makes it possible to influence which version is selected.

The version should preferably be written in the SemVer standard or similar format consisting of numbers and dots. Other formats pose risk of incorrect ordering.

Ultimately, the software selection process determines which version to pick, not knowing which versioning scheme the chosen software product uses. It is therefore best practice to specify the version in the [software configuration scripts](#) in the SemVer or adjacent formats.

OS is statement

The **OS is** statement can generally be used as the standard **is** statement, but it provides optional extension to specify the version range. This makes it possible to specify broad operating system, specific OS version, or version range for the specified OS. The **OS is** statement has 3 available variants as shown below.

```
OS is Os_name;
OS is Os_name-Os_version;
OS is Os_name Software Version_hints*;
Version_hints = ( on | above | from | below | to )
Os_version
```

Figure 8: VSDL-lite node OS is statements

The versioning keywords work in the same way as in the [software statements](#). Main difference is that the versioning constraints are applied to OS version instead of software version.

Vulnerability statement

The last statement on node is the suffer statement used to describe the CVE vulnerability, which should affect the node. The inclusion of this statement may require the Internet connection to work properly, in case the specified CVE vulnerability was not yet cached. The CVE vulnerability should correspond to the entry in NVD

```
suffers from CVE_vulnerability;
```

Figure 9: VSDL-lite node suffer statement

3.1.2 Allowed values

The VSDL-lite has many statements that expect values in a specific format and satisfy other numerous criteria. To simplify checking process, we provide summarized allowed values to the most important attributes and value types as visible below.

Type	Text based					Numeric	
Attribute	type	flavor	basebox	OS	software	memory	disk
Allowed attribute values	compute	The flavors allowed by environmental limits	The base boxes allowed by environmental limits	The operating systems allowed by environmental limits	The softwares specified in software conf. scripts	Positive number followed by unit with SI prefix (KB, MB, GB, TB, PB)	
	router						
Value	User	Path	Id	Addr	Version	Num	
Valid values	string without white space characters or string enclosed in quotation marks ("str str")		name of node or network	Ipv4 address (192.168.1.1)	Version in SemVer format	Positive integer	

Table 2: Allowed attribute values

3.1.3 Comparison with original VSDL

The first format and the language in which the scenario designer describes the scenario is VSDL-lite, language based on VSDL. VSDL provides many useful keywords and was initially considered as the main input language. However, some VSDL features proved hard to implement and did not fully suit the program pipeline.

To better suit capabilities of our deployment solution, we modified VSDL and removed time based keyword fully. We extended existing keywords to allow for more precise configuration in aspects where the KYPO deployment platform had unused potential. The full comparison of VSDL to VSDL-lite is shown below.

Original VSDL	VSDL-lite
scenario Id [duration Num]	scenario Id
type is (compute same as Id)	type is (compute same as Id)
flavor is (mobile ... same as Id)	flavor is (mobile ... same as Id)
cpu is (equal to Freq ... same as Id)	cpu is (equal to Freq ... same as Id)
	memory is (equal to Size ... same as Id)
	basebox is Id
disk is (equal to Size ... same as Id)	disk is (equal to Size ... same as Id)
OS is (Id same as Id ...)	OS is (Id same as Id ...)
	OS is Id { (above below from to on) Version }*
mounts software Id	mounts software Id
	mounts software Id { (above below from to on) Version }*
exists user Id	exists user Id
	exists user Id with password Password
user Id can (read write execute) Path	user Id can (read write execute) Path
	user Id can sudo
contains (file directory) Path	contains (file directory) Path
	contains (file directory) Path owned by Id
suffers from Vuln	suffers from Vuln
bandwidth is (equal to BW ... same as Id)	bandwidth is (equal to BW ... same as Id)
gateway has direct access to the Internet	gateway has direct access to the Internet
addresses range from Addr to Addr	(addresses address IP ip) range from Addr to Addr
firewall blocks (port Num IP Addr)	firewall blocks (port Num IP Addr)
firewall forwards port Num to Num	firewall forwards port Num to Num
firewall forwards IP Addr to Addr	firewall forwards IP Addr to Addr
node Id (is connected has IP Addr)	node Id (is connected has IP Addr)
[switch (on off) at ...] → ...	
	is user accessible
	IMPOSSIBLE

Table 3: VSDL and VSDL-lite keywords comparison

Dropped statements

Some statements were dropped, or their functionality was removed. The time related statements have been removed completely. The scenario no longer accepts duration as a variant, and the switch on and off statements were removed altogether.

Statements with changed meaning

The statements which could not be easily implemented in the chosen deployment platform but are still valid syntax and could potentially be implemented in future are: cpu is, bandwidth is, and gateway has direct access to the internet.

The flavor property was also modified to use KYPO flavors, and the type was relegated to differentiating between routers and hosts. These are still valid statements, but their behaviour may differ from other VSDL implementations.

Extended statements

Certain statements were extended to provide optional behaviour. Others had new operators added. The aim of optional extensions is to increase the expressiveness of VSDL-lite for advanced users, without complicating syntax for new users.

Comparative keywords used in the **is** statements such as equal to, bigger than, smaller than, etc. were extended by **at most** and **at least**. The **is** statement was also extended by base box and memory properties.

The ability to influence software and OS version was also improved. Version hints provide the way to define the interval of versions to which the software or OS must be matched.

The user related statements were also extended, by adding statements, which give user administrator privileges or ownership of file or directory. The last modification to the user statements is an optional extension to choose the user password.

Added statements

Only 2 new statements were added, the **is user accessible** for the networks and **IMPOSSIBLE**. The **is user accessible** is an optional statement added to allow designer to make the network user inaccessible when desired.

The **IMPOSSIBLE** keyword was added to allow better manipulation with logical statements. It is always evaluated as False and is used by the [specifier stage](#).

3.2 Preprocessed VSDL-lite

The preprocessed VSDL-lite is a standardized form of VSDL-lite syntax that follows a stricter format to improve machine readability in the next stage.

3.2.1 Standardized formatting

The most obvious difference from standard input is in the formatting. The preprocessing leaves the VSDL-lite definition in standardized form. Statements spanning multiple lines are merged into one line. The keywords are separated by a single space, and the default scenario keyword is added if not already present, as seen below.

```
scenario generated-game {
node desktop {
    OS is kali ;
}

network LAN {
    wifi-Router is connected ;
    desktop is connected ;
}

router wifi-Router {
    type is router ;
}
}
```

Figure 10: Snippet of preprocessed VSDL-lite scenario

3.2.2 Sub-statement bracketing

The preprocessed VSDL adds brackets inside the logical statement to simplify parsing in the [specifier stage](#). The added brackets ensure that during each step only one logical operation needs to be handled.

```
#raw input
not OS is windows or OS is windows and memory is
    larger than 4 GB;
#preprocessed
( not ( OS is windows ) ) or ( ( OS is windows ) (
    memory is larger than 4 GB ) ) ;
```

Figure 11: Preprocessor generated brackets

3.2.3 Vulnerability expansion

The last difference between the raw VSDL-lite scenario and the preprocessed one is the expansion of vulnerability statements. This is the most complex task during preprocessing and it also results in the largest difference between scenarios.

The suffer statement is replaced by a large composed statement consisting of sub-statements representing vulnerable configurations assigned to the vulnerability.

```
#raw input
suffers from CVE-2015-0241;
#preprocessed
( ( ( mounts software postgresql below 9.0.19 ) or (
...
) or ( ( OS is debian_linux from 7.0 below 7.1 ) or (
    OS is debian_linux from 8.0 below 8.1 ) ) ) ;
```

Figure 12: Preprocessor vulnerability expansion

3.3 SMT specification

The SMT specification is written according to the SMT-LIB standard version 2.0. The exact format of the generated specification consists of 5 segments as listed below.

1. scenario properties
2. scenario assertions
3. implication assertions
4. environmental properties
5. environmental assertions

The segments consist mostly of term declarations and assertions. The scenario and environmental properties declare constant terms to be used by their respective assertions.

The assertions are statements for which the [solver](#) tries to find a configuration that all assertions are true. The implication assertions use the implication logical operator to ensure that the implicated statements are true if their predicate is.

```
(declare-const users_node_desktop (Set String))
(declare-const password.admin_node_desktop String)
(assert(= "kali" password.admin_node_desktop))
(assert(=> (= "kali" password.admin_node_desktop) (
  set.member "admin" users_node_desktop)))
```

Figure 13: Snippets from SMT specification

3.3.1 Term names

The constant terms represent the variables that the [solver](#) changes during the solving process. They consist of the scenario and environmental properties.

The names of constant terms follow a simple formula, from which the data type and object association can be deduced. The term name can be divided into three parts:

- attribute - signifies what property of object the term represents
- object type - specifies what sort of object is the term associated with
- object name - name of object the term is associated with

The **object type** specifies whether the term is tied to the scenario object or to environmental limits. The **limit** and **getter** types belong to environmental objects, whereas the **node**, **network**, and **scenario** originate from the scenario definition itself.

The object name has dual usage. For terms associated with the scenario, the object name represents the name of the associated object. The environmental limits do not have instances of objects and generally do not use this field with the exception of the allowed limit type.

```
attr_type_name
attr0.attr1_type_name
attr0.attr1.attr2_type_name
```

Figure 14: Formula for term names

The **attribute** is composed of main attribute and optionally sub-attributes. The main attribute determines what the value means and what data type should be assigned to the term based on the definition of the term type in [term_types.py](#). The sub-attributes, if present, further categorize meaning of term, what user on node is the password tied to, and similar.

The attribute consists of one or more parts separated by dots. To allow names with special characters, the individual attribute parts are **encoded** before being concatenated. In the SMT specification, attributes are encoded to comply with SMT syntax requirements.

In the majority of program the term names are decoded to improve readability and simplify debugging process. The solver error caused by unsatisfiable constraints decodes the term names before printing the error message.

String encoding

The SMT-LIB standard 2.0 understandably restricts what characters can make up the constant term name. For this purpose, composing the term names from multiple attribute parts named by the designer creates an issue.

To mitigate this problem, V2K encodes the strings used in the term names. The strings are translated character by character, and all non-alphanumeric characters are translated to ASCII code and parsed to a number enclosed in acceptable special character as seen below.

```
#decoded string
mounted.zfs_storage_appliance_kit_node_PC1
#encoded string
mounted.zfs@95$storage@95$appliance@95$kit_node_PC1
```

Figure 15: Encoding process for term names

The special characters are parsed into numbers and enclosed between the symbols @ and \$. These symbols are acceptable within the term names and can be easily parsed by the decoder to obtain the original string.

The other special symbols present in encoded term names are **dots** and **underscores**. The dots are used to separate different attribute parts. The underscores are used to separate whole attribute, object type, and name.

3.3.2 Version encoding

Versions of software or operating systems are stored as strings. This avoids some issues with irregular versioning systems while still maintaining relative simplicity. To compare different versions, the string comparison is used. This introduces another issue of incorrect ordering, to fix this, the version undergoes special encoding and decoding process as seen below.

```
#decoded version
2.4.34
#encoded version
@2@4@@34
```

Figure 16: Encoding process for version strings

The versions are expected to be in the SemVer format, which does not have specified upper limit to version parts. This complicates the mapping of the

version string to a number. The V2K leverages the fact that the ASCII characters of the digits are ordered in the same way as the digits themselves.

The raw version is split into parts, and each part gets a prefix of the corresponding character length. This ensures that only numbers with the same number of digits are compared, otherwise the number with more digits is evaluated as bigger. The prefix uses the @ symbol, since it has a higher ASCII value than other numbers and is also used in string encoding.

3.3.3 Summary of statement to term relations

With many different statements and their variants, understanding which terms are related to which statements is important to fully understand the meaning of the SMT specification. To ease this process, we provide summary table detailing all statements from preprocessed VSDL-lite, and the terms generated from them.

Object type	Statement	Generated terms
Node (routers, hosts)	OS is OS_name ;	OS_node_Id
	Attribute is same as Id2 ;	Attribute _node_Id, Attribute _node_Id2
	Attribute is compare op Num Unit ;	Attribute _node_Id
	mounts software SoftName ;	mounted_node_Id
	mounts software SoftName {version hints} * ;	mounted_node_Id, mounted.# SoftName #_node_Id
	exists User ;	users_node_Id
	exists User with Password ;	users_node_Id, password.# User #_node_Id
	user User can sudo ;	users_node_Id, sudoers_node_Id
	user User can Right Path ;	permission.# User #.# Path #. Right _node_Id, directories_node_Id, files_node_Id, users_node_Id
	contains directory Path ;	directories_node_Id
	contains file Path/SubPath ;	directories_node_Id, files_node_Id
	contains file Path ;	files_node_Id
	contains file directory Path owned by User ;	files directories_node_Id, users_node_Id, owner.# Path #_node_Id
Network	IP range from Addr to Addr ;	netid_network_Id, netsize_network_Id, connected_network_Id
	node Id is connected ;	connected_network_Id
	node Id has IP Addr ;	connected_network_Id, connected.# Addr #_network_Id
	is user accessible ;	accessibe_network_Id
	gateway has direct access to the Internet ;	internet_network_Id
	firewall forwards IP Addr to Addr ;	forwards.IP.# Addr #_node_Id
	firewall forwards port Num to Num ;	forwards.port.# Num #_node_Id
	firewall blocks IP port ;	blocks. IP port _node_Id

Table 4: Relation table between preprocessed VSDL-lite statements and terms

The **bold text** signifies user-defined values, such as specific IPs, software names, and similar. The exception to this is a Pipe | operator, which is used when any of the keywords separated by the | symbol can be used. When in term names, the user defined values are encoded as marked with hashtag # symbol.

3.4 Solution

The solution holds the exact configuration chosen by the [solver stage](#) written in json format. The [solution.json](#) contains 4 entries, the name of scenario, routers, hosts, and networks, as visible below. The solution file provides a clear standardized structured description of the scenario infrastructure.

```
{
  "name": "generated-game",
  "networks": {
    "desktop-switch": {
      ...network properties...
    },
    ...
  },
  "hosts": {
    "Desktop": {
      ...host properties...
    },
    ...
  },
  "routers": {
    "Router": {
      ...router properties...
    },
    ...
  }
}
```

Figure 17: Simplified example of solution.json

The networks, hosts, and routers are populated by entries of their respective objects. The device, router, or network is then made up of properties tied to that object. The properties roughly correspond to attributes and constant terms from the [SMT specification](#). With the exception of more complex attributes, such as mounted software, user permissions, and firewall rules.

3.4.1 Software properties

The software configuration is described by the mounted and versions properties. The mounted represents softwares which should be installed on the host or router, and the version is a dictionary of software versions to be installed. In case the software does not have specified version, it is present only in the mounted property.

```

...
"mounted": [
    "http_server",
    "vim"
],
"versions": {
    "http_server": "2.4.34"
}
...

```

Figure 18: Snippet from the solution.json describing the software configuration

3.4.2 User permissions

The user permissions are the most complex property as they summarize the rights of users in regards to files and directories. They are represented as a dictionary of users and their sets of access rights.

The sets of access rights are themselves represented as dictionaries of file paths and allowed operations to those file paths. The allowed operations are stored as a list of allowed or disallowed operations as seen below.

```

...
"permissions": {
    "guest": {
        "documents/instructions.txt": [
            "r",
            "x"
        ],
        "documents/market strategy.txt": [
            "!r"
        ]
    }
    ...
}
...

```

Figure 19: Snippet from the solution.json describing the user permissions

The allowed operations can describe right to read, write, or execute encoded as r, w, and x respectively. To mark disallowed operation, the exclamation mark is added at the start.

3.4.3 Connected nodes

The connected is property of networks generated from connections terms based on **is connected** and **has IP** statements. The connected property in the solution is a dictionary of connected nodes and their assigned IPs as seen below.

The assigned IP is deduced from the assertions generated by the **has IP** statement or assigned at [solver stage](#) after the SMT assertions evaluation. The

result is a connected property with a unique IP specified for every connected node. The IP address are formatted in human readable format, which is the used in following stage.

```
...
"connected": {
  "desktop": "192.168.0.15",
  "router": "192.168.0.1"
},
...
```

Figure 20: Snippet from the solution.json describing the connected configuration

3.4.4 Firewall rules

The firewall configuration is described by 2 properties, the blocks and forwards. They both support 2 types of address, the IP and port. For each entry of specific rule and address type the affected network is added as **src** sub-property to later differentiate what traffic should be subjected to the firewall rule as visible in the example below.

During [the solver stage](#) the firewall rules are extracted from the networks and moved to the corresponding routers. The chosen routers function as the default gateway and therefore can be configured to implement required firewall rules.

```
...
"blocks": {
  "port": [
    {
      "dst": 22,
      "src": "192.168.0.0/24"
    }
  ],
  "ip": [
    {
      "dst": "146.190.62.39",
      "src": "192.168.1.0/25"
    }
  ]
},
...
```

Figure 21: Snippet from the solution.json describing the firewall rules

The blocks are stored as list of blocked entries, while the forwards are stored as dictionary of forward entries, with keys being the original destination address. All of the data types that make up the complex firewall properties are strings, with the exception of ports under the **dst** key.

The IP addresses are formatted in human redable format, same as with connected property, with slight deviation to the src sub-property which also includes the subnet mask to keep information about the network size.

```
...
"forwards": {
  "port": {
    "880": {
      "dst": 8080,
      "src": "192.168.0.0/24"
    }
  },
  "ip": {
    "213.81.150.11": {
      "dst": "195.49.188.162",
      "src": "192.168.1.0/25"
    }
  }
}
...
```

Figure 22: Snippet from the solution.json describing the firewall rules

3.4.5 Summary of properties

The properties of scenario objects are less numerous and more compact than the terms from which they originate. The compacting process may change the data type or group multiple terms together. To mitigate confusion, we provide a data type table with data type of different properties.

Data type	Network property	Host property	Router property
Integer	bandwidth	memory, disk, cpu	
String	netid	OS, basebox, flavor, type	
List of strings		directories, files, mounted, users, sudoers	
Dictionary of Strings	connected	owner, versions, password	
Bool	internet, accessible		
Other		permissions	permissions, forwards, blocks

Table 5: Types of generic limits

The table describes every property with exception of the scenario name. Most properties can be easily mapped to the terms with the same attribute name. The string properties are relatively straightforward with the exception of **netid** property which combines netid with netsize.

The host and router dictionary properties are following the same archetype as the versions. They assign information to entries from the list properties. The password specifies passwords belonging to the users and the owner, specifies which user owns the file or directory.

3.5 Sandbox definition

The sandbox definition is the final format produced by the last stage. It is designed to be deployed on KYPO CSC and CRP. The sandbox definition follows KYPO standard and is often times immediately deployable without further modification as long as program is correctly configured and has environmental limits set according to configuration on used deployment platform.

3.5.1 Topology definition

The sandbox definition consists of the topology definition and the provisioning definition. The topology definition generation is well covered by the V2K and most of it can be directly described in the VSDL-lite input, with the exception of few optional properties. The generated topology definition is written into **output**/topology.yml ready to be deployed.

3.5.2 Provisioning definition

The provisioning definition is more complex and consists of multitudes of files and folders organized in such a way as to enable scenario deployment with as little as 3 commands after generating scenario. The provisioning definition is located in **output**/sandbox/provisioning/.

The provisioning definition follows the standard file structure as a standard KYPO provisioning definition. The `group_vars/`, `host_vars/` and `playbook.yml` are only dependent on the names of scenario nodes and generally do not need to be touched.

Tasks and Vars

The directory `roles/` contains folder for each device defined in the scenario. Each of these folders can contain `vars/main.yml`, `tasks/main.yml`, and `files/...`. The variables contain relevant device properties, such as lists of users and their passwords. The firewall entries and similar. These are the used by tasks, which execute looped ansible scripts for on these variables.

The variables and tasks are generated only for non empty properties, based on device properties from the solution. The tasks are generated in a fixed order from most safe tasks, with a very small chance of failing to more volatile tasks. The simplified order is as follows:

1. base box specific tasks
2. user generation
3. sudoer generation
4. directories, files generation
5. modifying permissions
6. unversioned software configuration
7. versioned software configuration
8. firewall rules

Files

Lastly, the files/... contain files or directories as specified in the solution. During the provisioning process, the files are copied to the device. This makes it easy to modify the contents of the files, without complicating the scenario design process.

4 Program characteristics

The V2K at a foundational level is a simple program designed to translate one format of file to the other. Its behaviour can only be influenced by randomness, adding options when launching the program, or modifying [configuration files](#).

The translation process is done through 4 separate stages, to enable complex debugging on each stage, and to give designer a way to modify the scenario in any of these stages if needed. The separation of stages is mainly done to separate different tasks. This simplifies each stage and presents a simpler way to modify program on a limited level.

Influencing the program behaviour through [configuration files](#) is complex form of influencing

4.1 Program options

The program options provide way to modify program behaviour on basic level. The options are passed to the program as program arguments. The options are evaluated in the main python script before any of the stage are called. The user can easily get access to list of available options by running following command:

- `main.py --help`

With the **--help** option, the program will forego any stages and only print manual to the user. The other case in which the program foregoes using any work on the input file is if there is an invalid option, in which case the program prints out the error and recommends using **--help** to the user.

If all options are valid, the stages are called and executed in ascending order, from stage 1 to stage 4. In case of error, the translation stops and the error message is printed out. The cache and possibly the output directory will contain the results of all successfully completed stages.

These are the valid options:

1. `-i <input file>` : changes the input used by program to specified input file
2. `-o <output folder>` : changes the output directory to specified path
3. `-stage <start:end>` : inclusively specifies which stages to run
4. `--silent` : disables all non file output of program
5. `--help` : prints out the available options

4.1.1 Running program without options

If an option is not specified, the program will use default values. The default input file is `input.vsd1` in the same folder as the program, the default output directory is the output folder in the same folder as the program. By default program will run all 4 stages and output messages about its action to the console. More examples of implicit behaviour are presented in the [next subsection](#).

```
C:\...\v1.0>main.py
preprocessor finished in 0.01570892333984375 s
specifier finished in 0.11128711700439453 s
solver finished in 0.38098716735839844 s
translator finished in 0.06376504898071289 s
total time elapsed: 0.5717482566833496

C:\...\v1.0>
```

Figure 23: Example of running the program without options

4.2 Implicit behaviour

The V2K is designed to simplify the scenario design process and remove unnecessary steps. For this purpose, many decisions in the program are optional and V2K automatically makes these decisions for the designer if no value is specified. These implicit behaviours are as follows:

1. if no input is specified, `input.vsd1` is used
2. if no output is specified, `output/` is used
3. if no scenario is specified, the scenario is named **generated-game**
4. The router with an unspecified IP address will be assigned the first IP address after the netid, as long as it is not used by other devices.
5. If no IP address is specified for connection, it is randomly selected from the pool of IPs of the network, excluding any IP addresses that were specified in that network, regardless of whether they were ultimately assigned.
6. if no owner of file or directory is specified, the ownership is given to root
7. by default, the generated directories give read and execute rights to users
8. by default, the generated files give the users read rights
9. the user passwords are same as usernames if no passwords are specified
10. firewall rules apply both to incoming and outgoing traffic in affected network

4.3 Use cases

The V2K provides many different features, useful when developing the cyber security game for KYPO platforms. The use of the KYPO and Ansible standards makes it a flexible and viable tool when designing scenarios, even outside of KYPO platforms. Some examples and ideas about what to use V2K for are listed below.

4.3.1 Automatic generation

The creation of the topology definition is a relatively simple process in the scenario design process. However, generating many variations of the same scenario with visible differences can be a time consuming process. The V2K covers the topology generation well and with provided configuration scripts provides an effective way to automatize sandbox definition generation.

The logical statements and randomization features of V2K make it easy to generate slightly different sandbox definitions. This feature is currently limited to standardized configuration (different users, filenames, ip addresses, etc...).

The underlying benefit is that one scenario definition can easily generate dozens or so different sandbox definitions. While automatizing software configuration is currently unsupported, the already implemented features make the sandbox definitions different enough to make cheating in scored cyber-security exercises ineffective and unreliable.

In general, the skills required to complete any of the generated cyber-security exercises are the same, while the actual commands and values differ. The generated differences do not decrease the usefulness of tutorials, but they make the step by step walk-troughs useless without understanding the steps.

4.3.2 Simple provisioning

The creation of the provisioning definition is one of the most complex parts of the scenario design process. The V2K generates provisioning definition for common configuration tasks, while relying on user defined scripts to fill in more complex software configuration.

The common generated provisioning tasks remove unnecessary menial work and accelerate the design process by using the standardized Ansible scripts for simple tasks. The advantage of this approach is a significant reduction in small typos that break complex Ansible scripts.

The generated provisioning is written in Ansible and is therefore also usable for automatizing software deployment on other platforms. Although V2K does not currently feature the ability to generate cyber exercises for non KYPO platforms, the provisioning definition can be easily extracted and used in other platforms.

4.3.3 Verifying scenario

The other important features of V2K are formal verification and environmental limits. Verification ensures that the majority of the logical errors are caught before deployment. This accelerates the design and testing process. Verification is applied at every step, with the exception of custom software configuration.

The [environmental constraints](#) make it significantly easier to verify whether the scenario is deployable on the target platform. The simple change of limit variables followed by the execution of stages 3 and 4 can quickly inform the designer about the feasibility of deploying the scenario on the target platform.

4.4 Time complexity in practice

The main mission of V2K is to improve the scenario design process. To allow for efficient development, the V2K is optimized to fail early and fast when possible. Separation of stages also ensures that long calculations are avoided if the preceding stages encounter an error.

The practical time taken to generate the sandbox definition is dependent on different aspects for each stage. In general, the [specifier](#) and [translator](#) will always be faster than the [solver stage](#). The [preprocessor](#) can always be the fastest stage, unless a new vulnerability is queried.

For practical purposes, it is expected that each vulnerability will add 6 seconds upon first mention and each software statement with version hints can add up to a half a second. Below is a heuristic approximation of time needed to generate sandbox definition.

```

Estiamted_time =
    10ms * N_statements +
    200ms * N_version_hints +
    6000ms * N_new_vuln

N_statements - number of statements
N_version_hints - number of version hints for software
                    with defined configuration scripts or preconf. on BBs
N_new_vuln - number of vulnerabilities mentioned for the
                first time

```

Figure 24: Heuristic estimation of time needed for generating sandbox definition

5 Limitations

The V2K is a useful tool for designing scenarios for cyber security exercises. However, it is not omnipotent. There are many limitations to be aware of when using V2K.

Some of the limitations have apparent solutions and may be removed in the near future; others, however, are more foundational. The debugging and performance limitations have clear ways of improving. The cross platform support and precise configuration do not have any known complete solutions.

The separation of stages makes addressing the limitations easier and whether it is through continued development of V2K, or modifications done by 3rd parties, the below-mentioned limitations can improve. Below are listed limitations, their effects on the user and their possible solutions.

5.1 Debugging difficulty

The hard to understand errors and difficulty in tracking which statements from input file are responsible for errors are one of the most obvious limitations of V2K, ability to debug. This limitation is mostly due to separation of stages and the use of different syntaxes. The most apparent example are the errors produced by the [solver stage](#), as they leave tracking and understanding which statements caused the error to the designer.

This limitation is currently addressed with mapping tables in documentation in conjunction with the use of intuitive constant term names, which can be easily tracked after some practice onto the original input statements. The attribute names are usually present in each stage and directly explain the meaning of a statement, a term name, or a property in the solution.

The possible improvement which would fix this limitation is to create a mapping system which tracks which statements generated which assertions. Unfortunately, the V2K currently does not implement such a system, but the modular architecture of V2K allows for relatively simple addition of this system and is subject of possible future work.

5.2 Configuring software

The VSDL-lite is too abstract for this purpose and more manual low level work is required. The V2K does not handle software configuration script generation, it relies on scenario designer to provide library of deployable base boxes already preconfigured or database of [software configuration scripts](#).

This limitation does not have an apparent solution available and is unlikely to be changed. The use of LLMs can assist the designer in finding required Ansible scripts or translating configuration commands into Ansible. The newly written software configuration scripts can then be used repeatedly. However, an initial effort is still required.

The possible solution would be to extend the syntax of VSDL-lite or use LLM automatically to generate vulnerable configuration scripts at demand. The use of artificial intelligence to automatize software configuration is a complex and rapidly changing field. Whether there will be an LLM suitable for this task remains to be seen.

5.3 Program performance

The other known limitation with a clear path to improvement is performance. More precisely, the performance of the [solver stage](#). Currently, the V2K is viable and useful tool for designing scenarios, the formal verification, while not all encompassing, is magnitudes of times faster than practical deployment.

Although the current performance is acceptable for current V2K use cases, the sub-optimal performance forbids the V2K to be truly rapid development platform with immediate response. With improved performance, the V2K could be used to verify the scenario in real time during the design process.

The solution is the use of different SMT solvers to significantly improve the solver's performance. The V2K assertions are only a subset of SMT commands. With introduction of proper optimization and correct assumptions, the performance can be significantly increased.

5.4 Inherited limitations

The limitations originating from chosen deployment platforms can be divided into limited network topology complexity and limited hardware support. These are foundational limitations and can not be removed without targeting a different deployment platform.

The network topology types supported by V2K are limited by the KYPO topology definition. Although the definition is sufficient to simulate common cyber security scenarios, the more complicated networks needed for cyber security scenarios focused on complex network vulnerabilities cannot be defined in practice with VSDL or even KYPO itself.

The use of virtual machines by KYPO significantly reduces the barrier of entry. However, it limits the designer's ability to influence the hardware present in the scenario. This limitation means that CVE's hardware configurations are invalid and never used.

5.5 Internet access

The V2K requires that the devices have access to the internet. This requirement can limit the range of designed scenarios, albeit, it is always possible to fully block all traffic from other devices individually, completely blocking internet access is not allowed, and internet access statements are ignored.

This requirement arises from an inconsistency in the provisioning order on different deployment platforms. The software configuration tasks may require internet access to download the packages and would fail without internet access.

The V2K fully ignores internet access statements, as they are rarely needed, and can usually be substituted with firewall rules. Firewall rules can also break software configuration, but only a specific subset of firewall rules pose this risk. It is recommended to avoid blocking ports 80 and 443 on networks with devices that undergo software configuration.

This limitation could be easily removed, but that would require precise control over the provisioning process, which is significantly influenced by the deployment platform.

5.6 Linux specific

The other foundational limitation lies in cross platform capabilities. The more standardized and open nature of Linux platforms is the reason why the support for other operating systems, like windows and macOS is limited. While some of the generated Ansible tasks may run on non-Linux operating systems, the Linux operating system were prioritized in the development.

The V2K allows users to design scenarios with non-Linux nodes and in case the suitable base-box is known, the V2K will use it. However, the software configuration and portion of other provisioning capabilities are unlikely to work on these devices.

To fix this limitation, the [translator stage](#) would need a significant expansion to cover different operating system families, and software configuration would require a more complex system to interact with devices.

6 Setup process

The V2K is relatively simple lightweight program, requiring python 3, installed nvdlib and cvc5 together with an internet connection, when uncached CVEs are specified. After successful installation, the running program without options should print the [standard output text](#). To further configure program it is possible to configure environmental limits, base boxes, and software configuration scripts in the [data folder](#).

The V2K installation guide does not cover installation of KYPO CSC or KYPO CRP, as they are deployment platforms and are only needed to deploy the scenario, not generate it. The links for instructions on the setup process for deployment platforms are listed below.

- KYPO CSC [gitlab page](#)
- KYPO CRP [gitlab page](#)

6.1 Installation guide

To install the program, it is required to download or clone the repository from [github](#). Following this, the repository must be unzipped if necessary and moved to the desired location. Ensure the program maintains the structure as described in [Files structure](#).

The program can be installed in python virtual environment through use of venv and supplied required dependencies on python version 3.12, or directly on the system. Either way the command must be executed from the project directory.

- `pip install -r requirements.txt`

Alternatively it is possible to install dependencies directly as long as there are no dependency conflicts with existing libraries. This approach is simpler but may risk inconsistent behaviour, it is recommended to use python 3.12.

- `pip install pyaml`
- `pip install nvdlib`
- `pip install cvc5`

If problems arise, it is recommended to move to python 3.12 exact version, and use python library venv to create virtual environment on version python 3.12. To check active version of python use this command

6.2 Environmental limits

After successfully installing the dependencies, it is recommended to try to test the program, by running it without arguments. Start the [main.py script](#). This will make it use [default input](#), which is already provided and was downloaded with program.

The environmental limits are stored in the json file in the [data folder](#). They allow designers to add hardware and software constraints to the scenario generation process. The limits come in two types, generic limits and complex limits. The generic limits use the same attribute name as the keywords from VSDL and follow predictable syntax. The complex limits consist of base boxes and flavors.

6.2.1 Generic limits

The generic limits follow simple, predictable syntax. To apply a limit to a certain attribute, first create dictionary for the chosen attribute you wish to influence. The next step is to specify the type of limit and assign the limit value.

```
{
  ...
  "memory": {
    "individual.max":20000000000,
    "total.max":80000000000
  },
  "type": {
    "allowed": [
      "router",
      "compute"
    ]
  },
  ...
}
```

Figure 25: Snippet from environmental limits depicting generic limits

There are multiple valid operations for numeric attributes and one for text based attributes. The main purpose of the environmental constraints is to set hardware limits to ensure that the generated scenario is within these limits and can be actually deployed. See the full list of generic limit types below for further clarification.

Limit type	Target attributes	Value type	Condition added
individual.min	numeric	numeric	every instance of attribute is bigger or equal to value
individual.max	numeric	numeric	every instance of attribute is smaller or equal to value
total.min	numeric	numeric	combined sum of attribute properties is bigger or equal to value
total.max	numeric	numeric	combined sum of attribute properties is smaller or equal to value
allowed	string	list of strings	every instance of attribute must come from supplied list of allowed strings

Table 6: Types of generic limits

Generic limits can be applied to the disk, memory, OS, type attributes, and attributes without direct implementation such as cpu, and bandwidth. From these only OS and type are string attributes and can have allowed sets of strings specified. The disk, memory, cpu, and bandwidth attributes are numeric and can be constrained with minimum and maximum values individually or collectively.

6.2.2 Complex limits

Complex limits are used to influence the flavors and base boxes that are picked. They do closely mimic allowed limit type from generic limits, but introduce additional constraints tied to picking specific flavor or base box.

Flavors

The definition of complex limits constraining the flavor selection follows the allowed limit type from generic limits convention with one minor difference. Instead of value type being a list of strings, it is a list of dictionaries with properties of the chosen flavor.

The properties described are number of vCPUs, RAM and disk as seen below. The solver stage ensures that the chosen flavor has properties that match the device properties. The RAM corresponds to device memory and disk to disk of the device.

```
{
  "flavors": {
    "allowed": {
      "csirtmu.tiny1x2": {
        "vCPU": 1,
        "RAM": 2000000000,
        "disk": 20000000000
      },
      ...
    }
  }
}
```

Figure 26: Snippet from environmental limits depicting flavor complex limits

Base boxes

The definition of complex limits constraining the base box selection is using the allowed limit type from the generic limits convention. There are no differences in the allowed definition written in the environmental [constraints file](#) as seen below.

The main difference and reason base box limits are complex limit type is their implementation, which requires additional configuration for each allowed base box in the [base_boxes.yml](#) file.

```
{
  "base_boxes": {
    "allowed": [
      "ubuntu-22.04",
      "kali"
    ]
  }
}
```

Figure 27: Snippet from environmental limits depicting base box complex limits

Similarly to flavor selection, base box selection is tied to other device properties, such as the operating system. The solver ensures that the selected base box has the same operating system as the device.

6.3 Base boxes

The properties of the base boxes are stored in an yml file in the [data folder](#). The actual base boxes are images of operating systems loaded during the deployment process and deployed to virtual machines to simulate real devices.

The purpose of the `base_boxes.yml` file is to inform the V2K about the details of the base boxes, so it can properly modify the topology definition and handle software configuration. For the base box to be selected, it must be within the specified [allowed base boxes](#) configured under environmental constraints.

6.3.1 Base box properties

The base box entries contain basic information about the base boxes, such as the operating system present in the base box, the management protocol, and the management user. The base box entry can optionally contain the Ansible initiation scripts, which are then placed at the start of the provisioning tasks.

```
kali_vuln_HTTP:  
  OS: kali  
  mgmt_user: debian  
  mgmt_protocol: ssh  
  preconfigured_software:  
    http_server: 2.4.43  
  ansible:  
    - name: Update keyring  
      get_url:  
        url: https://archive.kali.org/archive-key.asc  
        dest: /etc/apt/trusted.gpg.d/kali-archive-key.asc
```

Figure 28: Example of base box entry in `base_boxes.yml`

Preconfigured software

The other important optional property from the base box entries is the dictionary of preconfigure software. These are the softwares which are already installed and configured on the image and do not need additional configuration.

This is one of two ways in which V2K handles software configuration; if a specific version of software is requested, the V2K may choose to pick the base box or image with the software version preconfigured and pre-installed.

This is a relatively fast option, as the installation and configuration process may take up valuable time and resources. Admittedly, preparing images with preconfigured software has its cost in both storage and effort needed to prepare such an image. The other approach is explained in the [software configuration scripts chapter](#).

From perspective of V2K, the base box and image are interchangeable terms. While base boxes are expected to have minimal configuration, it is not required.

6.3.2 Adding base box

Adding a new base box to V2K is a simple process, with the precondition that the base box is already working and usable by the deployment platforms. This can be a source of many headaches, V2K aims to minimize any originating from the inclusion process within the program itself.

The process consists of the following steps:

1. find working base box
2. add base box entry to `base_boxes.yml`
3. specify OS version, management user and protocol
4. optionally specify preconfigured software versions, if desired
5. optionally specify Ansible scripts to run at the start of provisioning
6. add base box to allowed base boxes in `limits.json`

Specifying preconfigured software

To specify preconfigured software in the base box, ensure that the base box has `preconfigured_software` property and that the entry under software name is the corresponding version in `semVer` format.

Hybrid approach

There are, however, downsides; the preconfigured software is an optional property for a reason, even if the software version is installed, it is not always desirable to specify it as preconfigured. In hybrid approach, the software is pre-installed but configured at runtime. This avoids slow download and build process, while still keeping the configuration flexible.

It is required to omit software from the `preconfigured_softwares` property. Otherwise, the configuration script will be skipped and V2K will assume that the software is already configured. The last step needed to ensure the hybrid approach works is to modify the configuration script supported platforms.

6.4 Software configuration scripts

The last file in the [data folder](#) is `softwares.yml`. It stores configuration scripts for different software versions. The role of software configuration scripts is to provide more precise control over the software configuration.

The software configuration scripts are stored as a dictionary of softwares, where each software has an assigned list of configuration scripts.

The individual configuration script as shown below contains 2 mandatory properties; the version and the Ansible scripts used to configure the software. They also contain properties to limit to which devices the Ansible script is applicable.

```

http_server:
  - version: 2.4.34
    base_boxes:
      - munikypo/ubuntu-22.04
    OS:
      - ubuntu-22.04
    ansible:
      - name: Install required packages
        apt:
          name:
            - build-essential
            - libpcre3
          ...

```

Figure 29: Snippet of vulnerable Apache 2.4.34 software configuration script in softwares.yml

6.4.1 Selection process

They also contain supported base boxes or supported operating systems, or both. The version and supported operating system or base boxes are used when selecting the software configuration script.

For the configuration script to be selected, the basebox or operating system must of configuration script must be configured on the target device. The other requirement is the matching version. The last requirement is that the device do not have desired software version already preconfigured.

After the software configuration script is selected, the Ansible configuration scripts are copied and inserted into the provisioning definition at the end of tasks.

7 Example of usage

The scenario design process is complicated even when using familiar tools. The V2K has many unique features and workflows that differ from the standard scenario design process. To better familiarize V2K users with the workflow, we provide an exemplary use of V2K to design the scenario below.

7.1 Writing scenario

The chosen scenario is a red team game focused on exploiting vulnerable apache server and launching a denial of service attack. The scenario takes place inside company, where disgruntled employee harms the availability of services as a revenge.

The scenario has 2 hosts, the server and the desktop, each on different network connected to the same router. The chosen operating systems are kali for the router and desktop, while the server has ubnutu. The scenario definition is listed below.

```

scenario DOS-apache-test{
node desktop{
OS is kali;
    exists user admin with password kali;
    user admin can sudo;
    exists user guest;

    contains file "documents/market strategy.txt" owned by admin;
    not user guest can read "documents/market strategy.txt";
    contains file documents/notes.txt;
    user guest can read documents/notes.txt;

    mounts software nghttp2-client;
}

node router{
    type is router;
    OS is same as desktop;
    memory is larger than 1GB;

    exists user admin;
    user admin can sudo;
}

node server{
    exists user admin;
    user admin can sudo;

    OS is ubuntu;
    suffers from CVE-2020-11993;
}

network server-switch{
    IP range from 192.168.1.0 to 192.168.1.127;
    node server has IP 192.168.1.15;
    node router is connected;

    firewall forwards ip 213.81.150.11 to 195.49.188.162;
    firewall blocks ip 146.190.62.39;
}

network desktop-switch {
    IP range from 192.168.0.0 to 192.168.0.255;
    node desktop has IP 192.168.0.15;
    node router is connected;

    firewall blocks port 22;
    firewall forwards port 880 to 8080;
}
}

```

Figure 30: The VSDL-lite scenario definition of example

The desktop is a machine accessible by the scenario participant. It has 2 users, admin and guest, only guest is initially known to the participant. It has installed nghttp2-client and contains 2 files. One is an important document about the marketing strategy, to which the participant had his reading rights revoked, the other is the notes.txt that detail the story and motivation of the red team.

The router has minimal configuration and does not need to be directly interfaced with for scenario objectives. It showcases use of same as keyword and type to differentiate routers from other devices.

The server has a few statements that define its configuration. Its operating system is ubuntu and it suffers from CVE-2020-11993, which is a vulnerability on Apache web servers. This vulnerability allows for incorrect logging to different connection which can lead to race conditions and ultimately crash of the web server. For the purposes of this example, we will focus only on denial of service.

The networks specify the IP addresses of both the server and the desktop in their respective networks, while relying on [implicit behaviour](#) to choose the IP address of the router. The firewall statements block ssh access to the desktop and redirect port 880 to port 8080 used by the web server.

The example scenario is intended as a showcase of different V2K capabilities, not as the most realistic or polished cyber security exercise.

7.2 Compiling scenario

To compile the scenario, the user must launch the program via the command line with the necessary arguments. We intend to fully generate the sandbox definition from the scenario definition. For this purpose, only the [options](#) that we need to specify are the input and output. Below is the result of failed attempt to compile scenario

```
C:\..\Path..\v1.0>main.py -i input.vsd1 -o output
preprocessor finished in 0.013306617736816406 s
specifier finished in 0.042793989181518555 s
Traceback (most recent call last):
.....
Exception: failed to find valid configuration for terms:
and assertions:
[(let ((_let_1 (= "Unknown" "Software")))) (let ((_let_2 (
  and _let_1 (= "Software" "ubuntu_linux")))) (let ((
    _let_3 ..... "enterprise_manager_ops_center")) (and
    _let_1 (= "Software" "http_server")) (and _let_1 (= "
    Software" "debian_linux")) (or _let_2 _let_2 _let_2) (
    and _let_1 (= "Software" "clustered_data_ontap")))))] !

failed at stage: solver !

C:\..\Path..\v1.0>
```

Figure 31: The failed attempt to compile the example scenario

As visible in the output above, the scenario compilation process will fail with an unclear error message. The cause of the error is an unavailable software configuration for any of the vulnerable configurations belonging to CVE-2020-11993.

This is probably the worst, least legible error the designer can get from VSDL. The complex and verbose assertion is the result of a complex nested statement that describes all configurations of the CVE vulnerability. To fix this error, the designer must provide [software configuration script](#) or [base box](#) with a preconfigured version of the software.

The example scenario has software configuration script available for the http server version 2.4.34 on ubuntu 22.04. In case of a software configuration script that is unavailable, see [Writing the software configuration script](#) to learn how to handle the situation. Once the software configuration scripts for the stated version range are available, the V2K will succeed in the solving process and generate the sandbox definition in the output folder.

```
C:\...\Path...\v1.0>main.py -i input.vsd1 -o output
preprocessor finished in 0.0 s
specifier finished in 0.06414341926574707 s
solver finished in 0.44545674324035645 s
translator finished in 0.054574012756347656 s
total time elapsed: 0.5641741752624512
```

```
C:\...\Path...\v1.0>
```

Figure 32: The successful attempt to compile the example scenario

7.2.1 Writing the software configuration script

When software configuration scripts are needed, there are multiple ways to obtain them, if the designer has access to library of Ansible scripts for cyber security scenarios the process is as simple as copying and inserting them into the database in [softwares.yml](#) as explained in [Software configuration scripts](#).

In case we need to write software configuration script ourselves, it is recommended to remove problematic statement first, see the preprocessed VSDL-lite definition, and find what version is needed. After that, recompile the scenario without the currently unsolvable suffer statement and deploy it locally.

If the scenario is too complex to deploy locally, isolate only devices that need to have a custom software configuration and deploy only them as explained in [Launching sandbox definition on KYPO CSC](#).

Once the scenario is deployed, log into the target device via vagrant ssh. Find the required configuration commands for custom software configuration through multiple attempts and the help of external documentation, forums, or LLMs.

Once you have required configuration commands, they must be translated to Ansible, this can be relatively efficiently done by the LLMs or manually with the help of the Ansible documentation. The result is a list of tasks that can be entered into the database in [softwares.yml](#) as explained in [Software configuration scripts](#).

```

- name: Install required
  packages
  apt:
    name:
      - build-essential
      - libpcrc3
      - libpcrc3-dev
      - libssl-dev
      - libexpat1-dev
      - wget
      - libnghttp2-dev
    update_cache: true
    state: present

- name: Ensure build directory
  exists
  file:
    path: "/usr/local/src"
    state: directory

- name: Download Apache source
  get_url:
    url: "https://archive.
    apache.org/dist/httpd
    /httpd-2.4.34.tar.gz"
    dest: "/usr/local/src/httpd
    -2.4.34.tar.gz"

- name: Extract Apache source
  unarchive:
    src: "/usr/local/src/httpd
    -2.4.34.tar.gz"
    dest: "/usr/local/src"
    remote_src: yes

- name: Download APR
  get_url:
    url: "https://archive.
    apache.org/dist/apr/
    apr-1.6.3.tar.gz"
    dest: "/usr/local/src/apr
    -1.6.3.tar.gz"

- name: Extract APR
  unarchive:
    src: "/usr/local/src/apr
    -1.6.3.tar.gz"
    dest: "/usr/local/src/httpd
    -2.4.34/src/lib/"
    remote_src: yes

- name: Rename APR directory
  command: mv apr-1.6.3 apr
  args:
    chdir: "/usr/local/src/
    httpd-2.4.34/src/lib"
    creates: "/usr/local/src/
    httpd-2.4.34/src/lib/
    apr"

- name: Download APR-util
  get_url:
    url: "https://archive.
    apache.org/dist/apr/
    apr-util-1.6.1.tar.gz"
    dest: "/usr/local/src/apr-
    util-1.6.1.tar.gz"

- name: Extract APR-util
  unarchive:
    src: "/usr/local/src/apr-
    util-1.6.1.tar.gz"
    dest: "/usr/local/src/httpd
    -2.4.34/src/lib/"
    remote_src: yes

- name: Rename APR-util
  directory
  command: mv apr-util-1.6.1
  apr-util
  args:
    chdir: "/usr/local/src/
    httpd-2.4.34/src/lib"
    creates: "/usr/local/src/
    httpd-2.4.34/src/lib/
    apr-util"

- name: Download Openssl 1.1
  source
  get_url:
    url: "https://www.openssl.
    org/source/openssl
    -1.1.1w.tar.gz"
    dest: "/usr/local/src/
    openssl-1.1.1w.tar.gz"

- name: Extract Openssl source
  unarchive:
    src: "/usr/local/src/
    openssl-1.1.1w.tar.gz"
    dest: "/usr/local/src"
    remote_src: yes

- name: Configure OpenSSL
  command: >
    ./config
    --prefix=/opt/openssl-1.1
    --openssldir=/usr/local/src
    /openssl-1.1.1w
  shared
  args:
    chdir: "/usr/local/src/
    openssl-1.1.1w"
    creates: "/usr/local/src/
    openssl-1.1.1w/
    Makefile"

- name: Build OpenSSL
  command: make -j{{
    ansible_processor_vcpus
    | default(2) }}
  args:
    chdir: "/usr/local/src/
    openssl-1.1.1w"

- name: Install OpenSSL
  command: make install
  args:
    chdir: "/usr/local/src/
    openssl-1.1.1w"

- name: Add openssl to library
  path
  copy:
    dest: /etc/ld.so.conf.d/
    openssl-1.1.conf
    content: "/opt/openssl-1.1/
    lib"
    mode: "0644"

- name: Refresh dynamic linker
  cache
  command: ldconfig

- name: Configure Apache
  command: >
    ./configure
    --prefix=/opt/apache-2.4.34
    --enable-so
    --enable-ssl
    --with-ssl=/opt/openssl-1.1
    --with-included-apr
    --enable-http2
    --with-nghttp2
  args:
    chdir: "/usr/local/src/
    httpd-2.4.34"
    creates: "/usr/local/src/
    httpd-2.4.34/Makefile"

- name: Compile Apache
  command: make -j{{
    ansible_processor_vcpus
    | default(1) }}
  args:
    chdir: "/usr/local/src/
    httpd-2.4.34"
    creates: "/usr/local/src/
    httpd-2.4.34/httpd"

- name: Install Apache
  command: make install
  args:
    chdir: "/usr/local/src/
    httpd-2.4.34"
    creates: "/opt/apache
    -2.4.34/bin/httpd"

- name: Stop system apache2 if
  present
  service:
    name: apache2
    state: stopped
    enabled: no
    ignore_errors: yes

- name: Set Apache Listen port
  lineinfile:
    path: /opt/apache-2.4.34/
    conf/httpd.conf
    regexp: '^Listen 80'
    line: >
      Listen 8080
      Listen 8443
      <VirtualHost *:8443>
      ServerName 192.168.1.15

      SSLEngine on
      SSLCertificateFile "/opt
      /apache-2.4.34/
      conf/server.crt"
      SSLCertificateKeyFile "/
      opt/apache
      -2.4.34/conf/
      server.key"

      Protocols h2 http/1.1
      DocumentRoot "/opt/
      apache-2.4.34/
      htdocs"
      </VirtualHost>

- name: Enable ssl
  lineinfile:
    path: /opt/apache-2.4.34/
    conf/httpd.conf
    regexp: '^#LoadModule
    ssl_module modules/
    mod_ssl.so'
    line: 'LoadModule
    ssl_module modules/
    mod_ssl.so'

- name: Generate certificates
  command: sudo openssl req -
    x509 -nodes -days 365
    -newkey rsa:2048 -
    keyout /opt/apache
    -2.4.34/conf/server.
    key -out /opt/apache
    -2.4.34/conf/server.
    crt -subj "/C=SK/CN=
    digital.sk"

- name: Enable http2
  lineinfile:
    path: /opt/apache-2.4.34/
    conf/httpd.conf
    regexp: '^#LoadModule
    http2_module modules/
    mod_http2.so'
    line: 'LoadModule
    http2_module modules/
    mod_http2.so'

- name: Make error log
  command: touch /opt/apache
    -2.4.34/logs/error_log

- name: Start custom Apache
  command: "/opt/apache-2.4.34/
    bin/apachectl start"

```

Figure 33: The Ansible script needed to manually download, install, configure and launch the vulnerable version of Apache http server

7.3 Modifying output

The generated sandbox definition is usually immediately deployable on KYPO CRP, but often times requires tuning to work on KYPO CSC. While the scenario can now be deployed, there are still influential steps available before deployment.

- modifying the provisioning definition to add tasks not covered by the V2K
- populating the files
- modifying the preconfig as further explained in [next subsection](#)

7.3.1 modifying the provisioning

The generated provisioning definition is located in the output folder under the sandbox definition folder. The benefit of V2K is the automatized standardized provisioning definition generation, which is generally less prone to errors. In case a finer configuration is needed, it can naturally be modified to accommodate any form of scenario deployable on KYPO platforms.

Frequent modification of the generated sandbox definition decreases the benefits of V2K. It is recommended to use software configuration scripts when certain precise configuration is repeatedly used on same service. This feature is intended for rare edge cases and should not be the norm when possible.

7.3.2 populating the files

The other case in which V2K allows and even expects the user to modify the generated sandbox definition is populating files. The V2K sandbox generation process does not specify or know what the contents of files are, and the VSDL-lite syntax does not support such features.

To modify contents of files, the user needs to open the generated sandbox definition, and in the sandbox/provisioning/roles folder navigate to modified device and its files. The files folder contains the copy of files to be copied onto the device after launching the sandbox definition.

This allows the user to add high fidelity, precise data to the scenario, without complicating the fundamental design process.

7.4 Launching sandbox definition on KYPO CSC

The sandbox definitions generated by V2K are expected to be primarily tested on CSC during the development phase. To this end, V2K generates sandbox definitions and topology definition in the same output folder. To launch and test the generated sandbox definition, following actions must be taken.

1. ensure CSC is installed and working
2. open command line interface in the output folder
3. execute command: **create-sandbox topology.yml**
4. fix the preconfig and vagrant file before continuing
5. execute command: **cd sandbox**

6. execute command: **manage-sandbox build**
7. wait for scenario deployment
8. test the scenario
9. **manage-sandbox destroy**

7.4.1 Problematic steps

The steps to launch the scenario and test can fail in some instances, if such occurrence happens, it can become impossible to execute **manage-sandbox destroy** and the devices can only be cleared with **vagrant destroy -f** command.

The most problematic step is the 4th step. While it is theoretically possible that no issue will arise, that is unlikely and guaranteed in this case. Especially as newer and newer versions of base boxes and operating systems are being simulated, while standard the deployment platforms and Ansible provisioning scripts remain on same version.

The common issue is that the base box does not support the Ansible or the used version. The other possible issue is a bad preconfig playbook, which expects the device to use ifupdown. In cases of newer ubuntu the ifupdown is replaced by netplan and that requires other pre-configuration playbook, such is the case with example scenario.

7.4.2 Modifying Vagrant file

In case of example scenario, the 7th step will likely fail on the server, without further modifications, the vagrant up command will use invalid preconfig generated by CSC and potentially encounter windows specific issues.

To ensure correct behaviour, we will need to modify the vagrant file by changing the precofnig playbook to customized preconfig playbook just for this scenario, we will name it **playbook_u20.yml** to signify its association with ubuntu. The other modification needed for KYPO CSC on windows is to add **custom parameters** to the virtual box provider we use.

```
...
# Device(host): server
...
device.vm.provider "virtualbox" do |vb|
  vb.memory = 2048
  vb.cpus = 1
  vb.customize [ "modifyvm", :id, "--uartmodel", "
    disconnected" ]
end
...
device.vm.provision "ansible_local" do |ansible|
  ansible.playbook = "preconfig/playbook_u20.yml"
...

```

Figure 34: The modified vagrant file

7.4.3 Modifying pre-configuration playbook

As ubuntu uses netplan as the network manager instead of ifupdown, it requires a different pre-configuration playbook. For purposes of this example scenario, we provide a working pre-configuration script. However, the provided script is applicable only to the example scenario and has hardcoded ip addresses.

```
---
- name: Router configuration
  hosts: routers
  become: yes
  tasks:
    - name: Enable IP forwarding
      sysctl:
        name: net.ipv4.ip_forward
        reload: yes
        sysctl_set: yes
        value: '1'
      vars: {}

    - name: Set up postrouting
      iptables:
        table: nat
        chain: POSTROUTING
        jump: SNAT
        out_interface: '{{ ansible_default_ipv4.
            interface }}'
        to_source: '{{ ansible_default_ipv4.address }}'

- name: Linux configuration
  hosts: ssh
  become: yes
  tasks:
    - name: Install net-tools
      apt:
        name: net-tools

    - name: Add aliases of all devices
      lineinfile:
        line: '{{ item.key }} {{ item.value }}'
        path: /etc/hosts
        with_dict: '{{ device_aliases }}'

    - name: Ensure netplan is installed
      apt:
        name: netplan.io
        state: present

    - name: Deploy netplan configuration
      template:
        src: roles/common/templates/netplan.yaml.j2
        dest: /etc/netplan/50-ansible.yaml
        mode: '0644'

    - name: Apply netplan config
      command: netplan apply

    - name: Remove old default route
      command: ip route del default

    - name: Add new default route
      command: ip route add default via 192.168.1.1 dev
        enp0s8

- name: Windows configuration
  hosts: winrm
  tasks:
    - name: Add aliases of all devices
      win_hosts:
        state: present
        canonical_name: '{{ item.value }}'
        ip_address: '{{ item.key }}'
        with_dict: '{{ device_aliases }}'

    - name: Controller configuration
      hosts: controller
      become: yes
      tasks:
        - name: Install pywinrm
          pip:
            name: pywinrm
            executable: "pip3"
        ...
```

Figure 35: The custom pre-configuration playbook

The `playbook_u20.yml` is located right next to the default generated pre-configuration playbook and uses the jinja template located in `output/sandbox/preconfig/roles/common/templates/netplan.yaml.j2`.

```
network:
  version: 2
  ethernet:
    {% for interface in routes %}
    {{ interface.interface_name | default('enp0s8') }}:
      addresses:
        - {{ interface.interface_ip }}/{{ interface.interface_prefix | default('24') }}

    {% if interface.interface_default_gateway is defined and interface.interface_default_gateway != '' %}
    routes:
      - to: default
        via: {{ interface.interface_default_gateway }}
    {% endif %}

    {% if interface.interface_routes is defined and interface.interface_routes | length > 0 %}
    routing-policy:
      {% for r in interface.interface_routes %}
      - to: {{ r.to }}
        via: {{ r.via }}
      {% endfor %}
    {% endif %}
  {% endfor %}
```

Figure 36: The jinja template used by the custom pre-configuration playbook

7.4.4 Accessing the scenario

Once the scenario is deployed, the user has direct access to every machine. The devices with GUI will be directly accessible in the virtual box and the remaining devices can be accessed with the vagrant ssh **device_name** command.

Unlike the KYPO CRP, the local deployment method is more open and can under certain circumstances provide advantage to participant which intends to exploit the CSC limitations. The CSC is therefore mostly recommended to be used on lower stake exercises and faster scenario development.

7.5 Launching sandbox definition on KYPO CRP

The launching process on KYPO is both easier and harder. Easier because the pre-configuration is generally already handled by better selection of base boxes and more complex pre-configuration scripts being used automatically. The complications arise from the barrier between the cloud platform and user.

7.5.1 Reconfiguring V2K for CRP

To generate sandbox definition deployable on the cloud, the base box selection needs to be modified to use base boxes available to KYPO CRP deployment platform. To this end it is useful to keep secondary limits.json file with environmental constraints specific to KYPO.

In case the KYPO CRP environmental limits are not already available the process to generate them is straightforward. Log into the KYPO CRP, navigate to images using sidebar and browse available images. Choose images from the available images and add them as base box entries into the V2K environmental limits and base box database in [data folder](#).

The [process of adding new base boxes](#) to be used by V2K is documented in previous section. It is important to choose correct management user and protocol, otherwise the KYPO deployment may fail in later stages. The other important step is to remove the CSC specific base boxes from base_boxes[allowed] complex environmental limit, otherwise the generated topology definition may still use now unavailable base boxes.

7.5.2 Deployment

Once the V2K is reconfigured to target environmental constraints on the KYPO CRP deployment platform, the next step is to re-run the V2K and generate a new CRP ready sandbox definition. Both KYPO and CSC variants of the vulnerable Apache http server scenario are available as examples, consisting of an input scenario, output folder, cache, and data folder used to environmentally constraint the sandbox generation.

With the valid sandbox definition generated, the next steps are as follows:

1. copy the topology into the sandbox folder
2. create gitlab repository
3. upload the contents of sandbox folder to the gitlab repository
4. open the KYPO CRP webpage and navigate to the sandbox definitions

5. create sandbox definition with gitlab URL and revision corresponding to the gitlab branch
6. create pool and assign the created sandbox definition to it
7. allocate pool
8. wait till the allocation succeeds or fail
9. in case of failure in the 1st or 2nd stage, ensure the correct base boxes are defined with corresponding management users and protocols
10. in case of failure in 3rd stage, check the output and determine what provisioning task failed, ensure that the base box undergoes required tasks before attempting complex task. (f.e. update keyring if you use apt later)

The aforementioned steps are basically the same as with any other KYPO sandbox definition. Although the provided examples should work out of the box, it is possible, that further updates or changes in repositories or Apache archives will change in future. In such a case the deployment would likely fail in last stage and it may be required to modify the provided example.

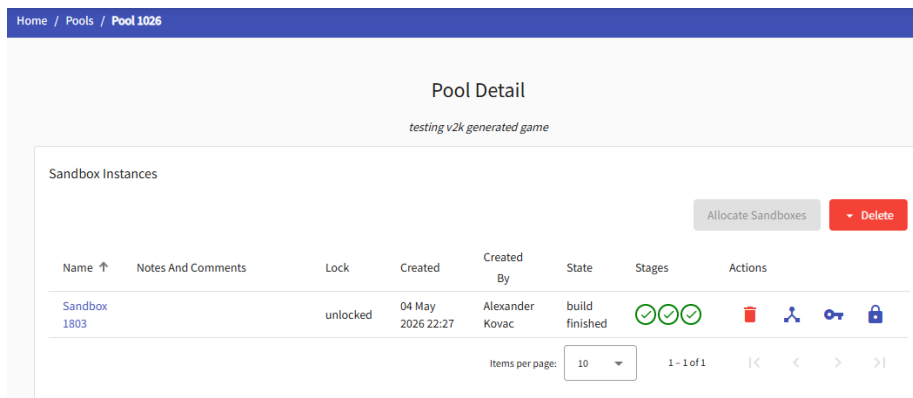


Figure 37: The successfully allocated pool with deployed scenario

The KYPO CRP is generally less useful for rapid development compared to the KYPO CSC, but its fine selection of images and more advanced pre-configuration significantly reduce the pre scenario debugging required.

7.5.3 Accessing the scenario

To access the scenario on KYPO the participants must be given ssh configurations to obtain access to the machines remotely, as required in the KYPO CRP deployment method. The SSH config can be downloaded directly from the KYPO portal by clicking on the [key icon](#) on the pool entry.

Once the participants, or designer who are testing out the cyber security game obtain the ssh config they can join the machines specified in the config. The SSH config can be used with ssh in the command below.

Some additional modification to the file may be desired. For example, changing the user to which the participant logs and possibly modifying the file paths of keys. Depending on the operating system used, there are further steps which may be necessary to use SSH config.

For purposes of this example, we will modify the User to be guest in SSH config, simulating the experience the red team actor, disgruntled employee would have.

```
C:\..Path.>ssh -F pool-id-..Digits..-sandbox-id-...
Hexadecimal...-user-config desktop
Warning: Permanently added 'proxy.kypo.fiit.stuba.sk' (
ED25519) to the list of known hosts.
Bad permissions. Try removing permissions for user: NT
AUTHORITY\Authenticated Users (S-1-5-11) on file
C:\..Path../pool-id-..Digits..-sandbox-id-...Hexadecimal
...-user-key.
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@ WARNING: UNPROTECTED PRIVATE KEY FILE! @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@

....
```

Figure 38: The failed attempt to connect to desktop via ssh

Modifying access rights

After downloading the SSH config and using it to connect to the desktop, the next obstacle will likely be the strict requirements for private keys. The ssh requires private keys to be inaccessible to normal users. On Windows, this requires modifying the file access rights. Below is an example of commands to do this on Windows, Linux can achieve the same results with the use of chmod.

```
C:\..Path.>icacls pool-id-..Digits..-sandbox-id-...
Hexadecimal...-user-key /inheritance:r
C:\..Path.>icacls pool-id-..Digits..-sandbox-id-...
Hexadecimal...-user-key /grant %USERNAME%:R
```

Figure 39: The commands to modify access rights on windows

Connecting via SSH

Once all of the steps are completed, the previous command will be successful and we will log in as the guest. After entering the password identical to the username, as the guest does not have a password configured, we can experience the scenario as intended. Below is an example of a successful ssh command


```

C:\...\Path...>ssh -F pool-id-..Digits...-sandbox-id-...
Hexadecimal...-user-config desktop
Warning: Permanently added 'proxy.kypo.fiit.stuba.sk' (
ED25519) to the list of known hosts.
Warning: ...Some Warnings...
guest@192.168.0.15's password:
Linux desktop 6.5.0-kali3-amd64 #1 SMP PREEMPT_DYNAMIC
Debian 6.5.6-1kali1 (2023-10-09) x86_64

...Some Disclosures...

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the
extent
permitted by applicable law.
Last login: Tue May 5 00:18:31 2026 from 100.100.100.186
+--(guest@desktop)-[~]
+--$

```

Figure 40: The successful attempt to connect to desktop via ssh

7.6 Ways to improve the example scenario

The example scenario is not very polished or realistic, and there are many possible ways to improve it. For inspiration to scenario designers and as a self-reflection and acknowledgement that our example is not flawless, we list some of the aspects in which the scenario could be improved.

7.6.1 Adding more realistic context

The V2K provides practically unlimited ability to populate files and provide high fidelity, realistic customized files to the scenario. This can be used to both fully populate the desktop with files expected to be found on highly used computer and provide server with well designed web page befitting that of organization mentioned in the scenario.

7.6.2 Making universal netplan pre-configuration playbook

The pre-configuration playbook on KYPO CSC is limited by use of static IPs. This was done to speed up development of the example scenario and is limitation, which could be turned into useful universal playbook to be used by other devices that use netplan as networking management.

Other possible improvements

There are many other ways to improve the example. Adding a third host, to monitor the web page availability and automatize the scoring. Installing more complex tools on the desktop to fully exploit the CVE vulnerability and cause the server to crash permanently. Or, providing detailed training definition. We leave this work to professional scenario designer.

7.7 Scenario walk-through

After connecting as guest, we will first check the situation on desktop. The /documents contains all files relevant to motivation of red team actor. Below are commands used to explore the files present on the desktop.

```
+- $ head /documents/market\ strategy.txt
head: cannot open '/documents/market strategy.txt' for
reading: Permission denied

+-- (guest@desktop)-[~]
+- $ head /documents/notes.txt
I am angry, the administrator(Josh) has sabotaged me.
I can't access marketing strategy document, nor can I
open the minv.sk.
I get redirected to mindop.sk.

I had nearly lost client today, because of this.
I know it's Josh, I will get my revenge, see how he
explains our server not working.

If you are reading this, know it was Josh who forced my
hand.
```

Figure 41: Commands exploring the documents

We can see the personal notes taken by the rouge employee and get the context of his personal motivation. The contents of these files are copied from the generated sandbox definition files and can be easily modified. The example is not intended to be the most realistic and polished scenario, but it shows the possibilities of V2K.

Following the first exploratory steps we focus on interacting with network, we check the minv.sk and the company's http server.

```
+-- $ curl www.minv.sk
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http
://www.w3.org/TR/html4/strict.dtd">
<html><head>
...
<p>The document has moved <a href="https://www.mindop.sk
/">here</a>.</p>
</body></html>

+-- (guest@desktop)-[~]
+- $ curl 192.168.1.15:880
<html><body><h1>It works!</h1></body></html>
```

Figure 42: Command curl used to check different web pages

From the results of the curl commands, we see that the desktop is forwarded to midop.sk when attempting to connect to minv.sk. This, combined with the previous blocked access to the marketing strategy files, shows the motivational context behind scenario actors.

The second curl shows that the company web server is working and can be accessed. We will use this fact to launch denial of service attack on the server. We will use the h2load library to flood the http server with requests. The result is webpage unable to reliably load during the duration of the attack.

```

+--(guest@desktop)-[~]
+-$ h2load -n 40000 -c 500 https://192.168.1.15:8443 >
    h2load.log 2>&1 &

+--(guest@desktop)-[~]
+-$ curl 192.168.1.15:880 --max-time 10
curl: (28) Operation timed out after 10002 milliseconds
    with 0 bytes received

+--(guest@desktop)-[~]
+-$ tail h2load.log -n 10

finished in 21.61s, 1789.64 req/s, 169.36KB/s
requests: 40000 total, 38688 started, 38687 done, 38667
    succeeded, 1333 failed, 1333 errored, 0 timeout
status codes: 38667 2xx, 0 3xx, 0 4xx, 0 5xx
traffic: 3.57MB (3746987) total, 1.23MB (1291962) headers
    (space savings 83.54%), 1.66MB (1740015) data
...
[1]+ Done h2load -n 40000 -c 500 https
    ://192.168.1.15:8443 > h2load.log 2>&1

```

Figure 43: The commands from DOS attack carried on Apache http server

The successful denial of service concludes the scenario. The red team got revenge and successfully attacked and damaged the availability of the web page. After completing the scenario, we disconnect from the desktop and proceed with the cleanup process. In case we are using KYPO CSC or graphical interface, we can skip this step.

```

+--(guest@desktop)-[~]
+-$ exit
logout
Connection to 192.168.0.15 closed.

C:\..\Path..>

```

Figure 44: The commands to exit ssh and leave scenario environment

7.8 Cleanup process

After the scenario is finished, it is important to release no longer needed resources. In case of KYPO CRP this requires using the graphical interface to delete sandbox instance by clicking on the [red bin icon](#).

In case of local deployment with CSC, this can be done with the following command.

```
manage-sandbox destroy
```

#alternatively if the previous command fails

```
vagrant destroy -f
```

Figure 45: The command to release resources on CSC